

Grounding AI Agents in Contracts: An Empirical Evaluation of Spec-Driven Test Generation

Michele Tufano, James McClure, José Cambronero, Runxiang Cheng, Sherry Y. Shi, Renyao Wei, Dorothy Chen, Franjo Ivančić, Livio Dalloro, Pat Rondon

{tufanomichele, jmcclure, jcambronero, chengsam, sherryshi, renyaow, dschen, ivancic, dalloro, rondon}@google.com
Google
USA

Abstract

LLM-based agents are increasingly used for coding tasks, where they have outperformed many classical approaches and scaled to repository-level tasks, such as test generation. However, when directly prompted to generate tests, these agents can fail to reason about the code and its underlying contracts, thereby missing edge cases and behavioral boundaries that affect test quality. To address this limitation, we propose *Spec-Driven Test Generation*, where we instruct an agent to first reason about – and explicitly document – code pre-conditions, post-conditions, and undefined behaviors. This intermediate semi-formal specification acts as a **cognitive scaffold** to guide subsequent test generation. Our evaluation on production bugs from Google shows that the spec-driven agent can deliver a 9.8 percentage points ($p = 0.0352$) improvement in bug detection rate and a 2.5 percentage point ($p = 0.0034$) improvement in branch coverage, compared to a traditional test generation agent baseline. Using LLM-as-a-Judge, we further show that test suites generated by the spec-driven agent are superior to the baseline and human-authored tests in 77.8% and 56.7% of the cases, respectively, and demonstrated improvements on following best practices, readability, and edge-case coverage.

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; **Empirical software validation**; **Specification languages**; • **Computing methodologies** → **Artificial intelligence**.

Keywords

Spec-Driven Development, Software Testing, Design by Contract, AI Agents, Large Language Models

ACM Reference Format:

Michele Tufano, James McClure, José Cambronero, Runxiang Cheng, Sherry Y. Shi, Renyao Wei, Dorothy Chen, Franjo Ivančić, Livio Dalloro, Pat Rondon. 2026. Grounding AI Agents in Contracts: An Empirical Evaluation of Spec-Driven Test Generation. In *Proceedings of the 1st International Workshop on Specification-Driven Development Life Cycle (SpecOps '26)*, October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3842652.3843195>



This work is licensed under a Creative Commons Attribution 4.0 International License. *SpecOps '26*, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2970-6/2026/10
<https://doi.org/10.1145/3842652.3843195>

1 Introduction

Large Language Models (LLMs) and autonomous LLM-based agents are rapidly reshaping the way developers write and test code. While agents are capable of generating many types of software artifacts, generating effective tests requires a deep understanding of the underlying codebase and its expected behavior in order to determine the correct output for a given execution state [5, 28]. Nonetheless, software engineers have long developed foundations to facilitate rigorous reasoning about expected program behavior. For example, Design by Contract (DbC) [27] explicitly documents behaviors in the form of pre-/post-conditions, and invariants.

Unfortunately, when agents are directly generating tests from existing code, there are no explicit contracts to anchor their exploration towards generating more effective tests. Agents can thus miss edge cases, hallucinate logic, and generate superficial tests that fail to systematically exercise the program’s state space.

To address this limitation and bridge the principles of DbC with agentic automation, we propose Spec-Driven Test Generation. Rather than relying on human engineers to define upfront contracts for existing systems, our methodology instructs an agent to *ground* its tests by first retroactively extracting and articulating a code component’s underlying contract before authoring any test code. The agent documents the pre-/post-conditions and undefined behaviors into a structured document that delineates inputs and outputs in natural language. Because this document blends the expressiveness of natural language with structure, but does not impose a strict mathematical formalism, we term this a *semi-formal specification*. This specification then acts as an oracle to encourage the agent to explore the state space more systematically.

To empirically evaluate whether this self-generated oracle can lead to superior test suites, we designed a comparative study evaluating a two-phase Spec-Driven Agent against an agent that directly generates test suites, which represents standard practice in agentic test generation. On a dataset of 90 historical production bugs from Google’s Issue Tracking System, we employ a “Greenfield Test Generation” setup to measure improvements in structural coverage and historical bug detection rates. Furthermore, we evaluate the semantic completeness of the generated specs using a novel *Contract Coverage* metric, which quantifies how a spec covers incorrect behavior of an evaluated bug. To assess test rigor, we employ a pairwise LLM-as-a-Judge that uses a large LLM (Gemini 3.1 Pro) that is better capable of evaluating the outputs generated by the agents that use a small LLM (Gemini 3 Flash).

Our findings demonstrate that the Spec-Driven Agent achieves a bug detection rate of 63.2% at $k = 5$ runs, which represents a

statistically significant ($p = 0.0352$) 9.8 percentage points improvement over the baseline, and a highly statistically significant 2.5 percentage points improvement in branch coverage ($p = 0.0034$).

Compared to the test suites generated by the baseline, a pairwise LLM-as-a-Judge rates the Spec-Driven-Agent-generated counterparts as overall superior in 77.8% of cases and have higher quality across all evaluated categories: adherence to testing best practices (65.6%), readability (68.9%), and edge-case coverage (83.3%). Compared to the original, developer-authored test suites, the Spec-Driven-Agent-generated counterparts are rated overall superior in 56.7% of cases, showing parity with human-level engineering rigor.

When we use our proposed Contract Coverage metric to analyze the generated specs, we find that this metric scales robustly, starting at 61.1% at $k = 1$ and reaching 78.9% at $k = 5$. This metric also represents a predictor of testing success ($p = 3.62 \times 10^{-14}$): when the generated spec successfully captures the violated behavioral contract, the resulting test suite detects the target bug in 54.9% of cases, compared to a mere 19.4% detection rate when the contract is missing from the generated spec.

In summary, this paper makes the following contributions:

- **Spec-Driven Test Generation Framework:** We propose a novel, two-phase agentic methodology that effectively improves the agent’s test generation ability by instructing agents to explicitly extract and document behavioral contracts prior to code synthesis.
- **Contract Coverage Metric:** We introduce a novel evaluation metric designed to measure the empirical coverage of AI-generated specifications and assess their capacity to capture violated business logic in real-world scenarios rather than claiming formal mathematical completeness.
- **Industrial Empirical Evaluation:** We provide a rigorous comparative analysis using a dataset of 90 historical bug-fixes from Google’s production systems, demonstrating that Spec-Driven Test Generation yields statistically significant improvements in both structural coverage and historical bug detection, while matching the qualitative rigor of expert, human-authored test suites.

Our findings suggest that treating semi-formal specification generation as a prerequisite step—rather than relying on direct code synthesis—is the key to unlocking agentic test generation capable of matching expert human engineering rigor. While this paper strictly evaluates the application of these behavioral contracts for agents, future work can explore the broader utility of these agent-generated specifications as valuable documentation and system alignment artifacts for human developers.

2 Semi-Formal Specifications for API Behavior

Specifications are widely used in software engineering and programming languages, and what exactly constitutes a specification can vary widely, depending on the ultimate goal and the intended audience (or tool). In practice, a program specification can range from a natural language description of intended behavior and associated properties to a rigorous mathematical representation of the expected behavior. In this work, the term “specification” means a semi-formal contract for an API. We are inspired by DbC and capture pre- and post-conditions for APIs. However, in our approach

these contracts are written in natural language and are assembled into a structured artifact (i.e., `.spec.md`) that will be used to reason about the underlying program during agentic test generation.

Formal Definition of the Specification Artifact. To operationalize this semi-formal paradigm, we define the specification artifact for a given source code component as a structured document consisting of specification blocks for every API (public and private functions, classes, methods) extracted from the code.

For any given code unit c , its behavioral specification S_c is formalized as the tuple: $S_c = \langle D_c, Pre_c, Post_c, Sug_c \rangle$

- D_c : A natural language description summarizing the code unit’s intended behavior.
- Pre_c : A set of pre-conditions that define the exact state or constraints required *before* execution.
- $Post_c$: A set of post-conditions that define the guaranteed state *after* execution. These encompass input validation (error handling), success values, state side-effects (e.g., database writes), and expected state variants.
- Sug_c : A local subset of test suggestions explicitly tied to enforcing the conditions in Pre_c or $Post_c$.

Crucially, each condition within Pre_c or $Post_c$ is intended to be evaluated against the existing unit test suite and assigned one of two verification states: (i) *Tested*: The condition is empirically verified by an existing test; (ii) *Untested*: The condition lacks verification, directly prompting a corresponding test suggestion in Sug_c .

Generating Specifications with AI Agents. Semi-formal specifications provide a valuable cognitive scaffold for AI agents. By forcing the LLM to systematically articulate pre-execution requirements and post-execution guarantees, the methodology imposes the structural rigidity necessary to constrain hallucinations and rigorously explore the state space. Simultaneously, expressing these constraints in natural language yields a format that is accessible for developer review, remains expressive and flexible, and preserves the option to seamlessly adapt to diverse downstream backend formalisms (e.g., Lean [13] or SMT-LIB [6]).

While traditional miners (e.g., Daikon or trace analysis) treat execution as ground truth—possibly formalizing buggy behavior as expected—LLM agents avoid only summarizing the existing implementation by attempting to infer true developer intent. Specifically, the agent can cross-reference local code with natural language docstrings, comments, existing tests, and caller-site API usage patterns across the repository to deduce implicit component expectations. Synthesizing a contract from these intent sources restricts semantic drift and preserves human-readability while generalizing (out-of-the-box) across diverse languages.

Illustrative Example. Consider a subset of a specification generated for a hypothetical `CustomerDatabase.add_customer` method. The agent analyzes the source code and existing tests to synthesize the semi-formal contract block shown in Listing 1.

In this example, the agent maps verified post-conditions to their corresponding test cases (✓). When it identifies a gap through code analysis—such as the implementation handling an empty name, but no test explicitly verifying this path—it flags the post-condition as

Specification: CustomerDatabase.add_customer

CustomerDatabase.add_customer Public Method
Adds a new customer to the backend, or raises an exception if there was an error.

Pre-conditions

- The database connection has been established (guaranteed by public API).

Post-conditions

Input Validation

- ✓ Raises `InvalidArgumentError` if `customer.phone_number` is invalid.
Tested by `add_customer_invalid_phone_test`
- ✗ Raises `InvalidArgumentError` if `customer.name` is empty.

Tip: Test that `InvalidArgumentError` is raised if the `customer.name` parameter is empty.

Success Scenarios

- ✓ Returns `None` if the customer was successfully added.
Tested by `add_customer_success_test`

Listing 1: Illustrative example of a semi-formal specification. The agent extracts pre-/post-conditions and checks whether they are already tested or missing (and suggests a test).

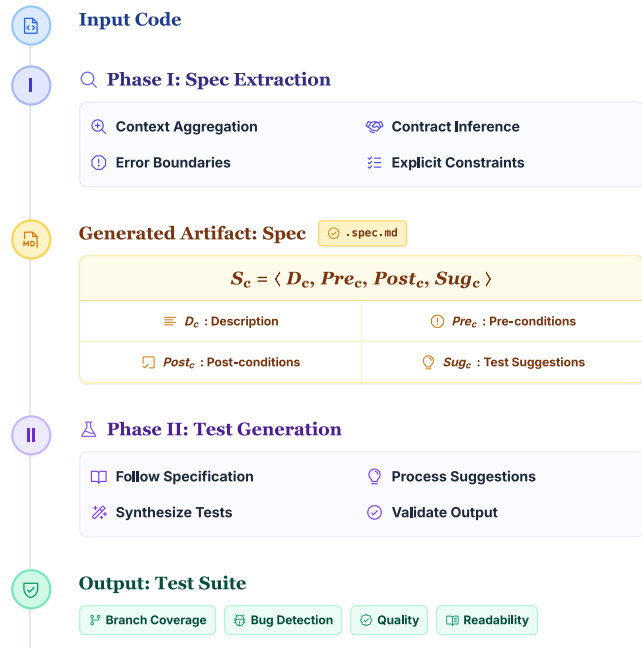


Figure 1: Overview of spec-driven test generation.

untested (X) and automatically synthesizes a structural test suggestion. This artifact serves as the blueprint reasoning plan for the subsequent test generation phase.

3 Proposed Approach

We now discuss our two-phase, agentic framework to generate specifications of the form described in §2 then generate tests. Figure 1 provides an overview of the framework.

3.1 Phase I: Specification Extraction

Given a target source file, optionally an existing unit test suite, and access to standard codebase navigation and editing tools, the agent’s objective in the first phase is to produce a specification document containing a tuple S_c for every unit of code c within the component (note: we do not programmatically enforce coverage but encourage it via the agent’s instructions). This phase proceeds through four steps:

- (1) **Context Aggregation:** The agent reads and analyzes the code under test—alongside any available natural language requirements, docstrings, inline comments, and protocol RFCs—to extract the structural hierarchy and intended business logic (D_c).
- (2) **Contract Inference:** The agent is instructed to systematically deduce the behavioral and safety boundaries for each method. It must explicitly formalize these limits by populating the pre- (Pre_c) and post-conditions ($Post_c$).
- (3) **Coverage Assessment:** The agent acts as an analytical cross-referencer, evaluating its inferred conditions against any (optionally) provided unit tests and actively discovering additional relevant tests across the codebase. It marks each condition in Pre_c and $Post_c$ as either *Tested* (✓) or *Untested* (X) based on current test assertions.
- (4) **Artifact Generation:** For every *Untested* condition, the agent generates a specific, targeted test suggestion to populate Sug_c . The final output of this phase is the collected, semi-formal specification document reflecting the complete $\langle D_c, Pre_c, Post_c, Sug_c \rangle$ tuples for all methods, including helper functions and private methods.

3.2 Human-in-the-Loop Specification Curation

Because the generated specification artifact is human-readable, our framework naturally supports an *optional* Human-in-the-Loop (HITL) step between the specification extraction and test generation phases. In this step, the developer can refine the inferred Pre_c and $Post_c$, as well as accept/reject/amend Sug_c based on their expertise, before any code is generated.

3.3 Phase II: Oracle-Driven Test Generation

Once the specification artifact is generated and optionally manually finalized, the agent transitions to the synthesis phase. Given the finalized specification S , and the current test suite T_{base} , the agent outputs an augmented test suite T_{gen} that implements the test suggestions (Sug_c) corresponding to the *Untested* conditions identified in Phase I.

By gating test generation behind the completion of S , the agent’s context window is primed with explicit boundary conditions. The agent systematically iterates through Sug_c to synthesize, build, and validate concrete unit assertions, producing a final augmented test suite grounded in behavioral contracts.

4 Empirical Design

To understand how this spec-driven framework can improve agentic test suite generation, we perform a comparative analysis using faults found in industrial production environments. Our study aims to answer the following research questions:

- **RQ1 (Quantitative Effectiveness):** To what extent does grounding an agent with a semi-formal specification improve structural coverage and the rate of detecting real-world, historical bugs?
- **RQ2 (Qualitative Rigor):** How do tests generated using and not using semi-formal specifications compare in terms of readability, best practices, and edge-case coverage?
- **RQ3 (Specification Accuracy):** How accurately do the agent-generated semi-formal specifications establish testing oracles that capture the violated behavioral contracts?
- **RQ4 (Cost-Efficiency):** What is the token consumption overhead incurred by the Spec-Driven Agent compared to a non-spec-driven, baseline agent?

4.1 Dataset Collection

We curated a dataset of 90 historical bug-fixes (pairs of buggy and fixed code) from Google’s Internal Issue Tracking System. Similar to foundational evaluation frameworks like Defects4J [23], we evaluate our approach on real faults because real-world production bugs inherently involve complex implicit state dependencies and boundary violations that stress an AI’s code reasoning capabilities far beyond standard generative tasks.

The dataset was strictly filtered to include only human-filed bugs with verified fixes and reproducible runtime failures. Bugs involving trivial syntactic fixes, multimedia processing, or personally identifiable information were excluded. Furthermore, we only consider bugs whose fix changes a single production code file, a corresponding test file, and optional BUILD configuration files. The final dataset spans multiple languages, including C++, Java, Python, and Go, providing evidence of language-based generalization of our findings.

4.2 Agent Configurations

To ensure a controlled comparative analysis, we isolate the impact of the specification reasoning phase by deploying both the baseline and spec-driven agents on the exact same underlying agentic architecture. Both agents use the Gemini 3 Flash [17] and operate within an identical agentic harness equipped with the following tool set: `view_file`, `list_dir`, `write_to_file`, `run_command`, `grep_search`, `find_by_name`, and a specialized `run_test` execution tool. Both agents use the default Gemini 3 Flash inference parameters: Temperature 1.0, TopP 1.0, and TopK 50.

Baseline Agent. The baseline is a standard prompt-to-code agent, broadly representative of existing architectures [11, 43]. It is provided with the source file under test and explicitly instructed to:

- (1) Analyze the code file.
- (2) Create a comprehensive test suite.
- (3) Modify the test file and optionally BUILD file if necessary.
- (4) Run the `run_test` tool with a provided test target to validate its generated implementation.

Spec-Driven Agent. Our proposed agent uses the same model and tool set but is bound by the framework described in §3 – Instead of direct code synthesis, the agent is forced to construct and validate the intermediate behavioral specification (S) before generating tests.

While the original framework supports HITL (§3.2), evaluating the efficacy of human intervention introduces significant subjectivity and confounding variables. Therefore, we bypass the HITL phase and accept all agent-generated test suggestions autonomously to facilitate a scalable, objective assessment of the model’s baseline reasoning capabilities.

4.3 Greenfield Test Generation Setup

To evaluate whether the semi-formal specification provide a “cognitive scaffold” for the agent to generate tests, we employ a *Greenfield Test Generation* setup, where (by construction) no prior tests are available for the to-be-tested code. This deliberate isolation prevents the agent from taking heuristic shortcuts—such as copying established test structures or performing simple test repair—and instead forces it to rely entirely on its fundamental code understanding and inferred specification to navigate the complex state space. In addition, completely removing prior test suites ensures a standardized evaluation baseline across projects of varying initial test quality.

For each bug b in our dataset, let C_b denote the source file in its original buggy state, C_f denote the source file in its fixed state, and T_b denote the pre-existing test suite source file(s). Note that within Google’s build system, T_b is easily retrieved by identifying all test targets that explicitly declare C_b as a direct dependency.

The evaluation procedure follows these steps:

- (1) **Initialize Fixed Source:** The agent’s workspace is synchronized to the state immediately following the human fix, establishing the clean implementation C_f from which expected component behavior is extracted. To prevent evaluation tautology (i.e., the agent reverse-engineering tests directly from the bug or fix diff), the agent is kept entirely blind to the bug, the commit message, and the fix diff during both specification and test generation phases.
- (2) **Strip Existing Tests (Greenfield):** To enforce a greenfield test generation scenario, we remove the existing test suite T_b , resulting in an empty test target $T_0 = \emptyset$.
- (3) **Agent Invocation:** Let $A \in \{A_{base}, A_{spec}\}$ represent the agent under evaluation. The agent is provided with the fixed source code C_f and the empty test target T_0 to generate a new test suite $T_{gen} = A(C_f, T_0)$.
- (1) **Validation and Coverage Assessment:** The test suite T_{gen} is executed against the fixed code C_f . In this step, we compute the structural coverage, denoted as $Cov_{line}(T_{gen}, C_f)$ and $Cov_{branch}(T_{gen}, C_f)$, and record whether the test suite passes on the correct logic (i.e., $Pass(T_{gen}, C_f) = \text{True}$).
- (2) **Fail-to-Pass Verification (Bug Detection):** The agent’s workspace is reverse-patched to the original buggy implementation C_b (except for the test suite and optionally modified BUILD files). The test suite T_{gen} is then executed against C_b . We formalize successful bug detection, $D(b, T_{gen})$, as true if and only if at least one individual test $t \in T_{gen}$ fails when evaluated on the buggy code:

$$D(b, T_{gen}) = \begin{cases} 1, & \text{if } \exists t \in T_{gen} \text{ such that } t(C_b) \rightarrow \text{Fail} \\ 0, & \text{otherwise} \end{cases}$$

Note that build failures are explicitly not considered as test failures and thus do not contribute to successful bug detection. The bug detection metric verifies that the generated test suite acts as a valid oracle, where the agent can recover the contracts and tests that successfully isolate and trigger the historical fault. The reverse-patching step mirrors the fail-to-pass protocols established in prior literature evaluating both classical [3, 36] and LLM-based [44] test generation tools against real-world defects.

4.3.1 Post-Execution Sanity Checks. To preserve the integrity of our evaluation, we employ post-execution sanity checks on the agents’ file operations. After each run, we audit the workspace trace and automatically discard (and do not repeat) any run where the agent modified the source code under test (C_f) or any other pre-existing source files within the repository. This validation step is critical: permitting the agent to alter the source code could inadvertently inject new faults into the fixed implementation, leading to test failures that are artifacts of the agent’s tampering rather than authentic detections of the historical bug (i.e., false positives). For a run to be considered valid, modifications to pre-existing files must be strictly limited to the target test file and, if necessary for compilation, the project’s build configuration files.

Nonetheless, to fully accommodate agentic reasoning, planning, and our proposed Phase I contract extraction, both the baseline and spec-driven agents are permitted to freely create and iterate on temporary auxiliary files during their run, such as markdown specifications (.spec.mds), task lists, and scratchpads.

4.4 Evaluation Protocol and Metrics

4.4.1 RQ1: Quantitative Effectiveness. To account for the inherent non-determinism of LLM, we perform $k = 5$ independent sampling runs for each bug configuration. We evaluate the generated test suites across three dimensions to ensure basic validity, breadth of coverage, and depth of bug detection:

Test Suite Pass Rate (pass@ k): measures the rate at which the entire generated test suite compiles and passes on the *fixed code*. Evaluated as pass@5, this metric serves as a sanity check before we assess the suite’s coverage or fault-finding depth.

Fault Detection Rate (detect@ k): measures the rate of successfully detecting historical bugs via Fail-to-Pass validation (i.e., whether *at least one* of the $k = 5$ generated test suites passes on the fixed implementation but fails on the reverse-patched buggy state). While software testing literature frequently relies on mutation scores, synthetic mutants often fail to represent the semantic complexity of real-world developer errors [36]. In this study, we conceptualize the historical bug as the “most valuable mutant,” making detect@ k our primary measure of test suite efficacy.

Structural Coverage: We check raw line and branch coverage.

Statistical Tests. We use the statistical estimation and hypothesis testing procedures below to assess the robustness of our results:

Estimation via Bootstrapping: For pass@ k and detect@ k metrics, we perform bootstrapping at the bug level (resampling the

90 unique bugs with replacement) with $n = 10,000$ resamples. This allows us to calculate the mean estimation alongside the 95% Confidence Interval (CI), providing a robust measure of effect size and result stability. This statistical estimation methodology is performed similarly to the task-level bootstrapping utilized in the evaluation of the OpenAI Codex model [10].

McNemar’s Test for Binary Detection: For comparing fault detection rates between the baseline and spec-driven agents, we construct a 2x2 contingency table of Baseline Success and Failure versus Spec-Driven Agent Success and Failure. We apply McNemar’s test to determine if the differences in detection rates are statistically significant with $p < 0.05$.

Wilcoxon Signed-Rank Test for Continuous Coverage: We use this paired difference test to compare line coverage distributions and branch coverage distributions, given that coverage metrics on paired agent runs are typically non-normally distributed.

4.4.2 RQ2: Qualitative Test Rigor via LLM-as-a-Judge. We employ an LLM-as-a-Judge to assess different qualitative dimensions of the generated test suites that structural metrics cannot capture.

To mitigate model capability risks from LLM-based assessment, we employ a stronger model (Gemini 3.1 Pro [18]) to judge the outputs of our agents that use Gemini 3 Flash [17]. To mitigate judging bias, we randomize the order of the two provided test suites and anonymize their origins by denoting them as “Test A” and “Test B” to the judge.

The judge considers four core qualitative criteria and outputs a detailed rationale and preference (“A”, “B”, or “Tie”) for each of the following dimensions:

Adherence to Testing Best Practices: To establish an objective standard, we instructed an LLM to condense Google’s internal developer documentation on unit testing best practices into a core set of principles, which were then provided to the LLM judge as an evaluation rubric. This condensed list emphasizes concepts such as behavior-driven testing (focusing on testing distinct behaviors and states rather than methods), maintaining logic-free assertions (prioritizing DAMP (“Descriptive and Meaningful Phrases”) over DRY (“Don’t Repeat Yourself”) to ensure tests are trivially correct upon inspection [41]), and utilizing proper factory helpers for complex object construction.

Readability and Structural Clarity: Descriptive naming conventions and adherence to the Arrange-Act-Assert pattern [7].

Edge-Case Coverage: Strategic targeting of boundary values and exceptional behaviors explicitly identified in the specification.

Overall Superiority: A holistic assessment determining which test suite is superior overall.

We perform dimension-wise majority voting over 5 LLM judge runs per agent run. If there is a tie or no strict majority is reached among the invocations, the final aggregated result for that bug and category defaults to a “Tie”. Finally, to quantify the variance and reliability of the LLM judge, we compute and report the agreement rate across the 5 invocations for each evaluation criteria.

Comparison to Original, Developer-Written Test Suites. To understand how our approach compares to expert engineers, we also perform the same LLM-as-a-Judge assessment to compare between the Spec-Driven Agent’s generated test suites and the original test suites authored by the human developers.

4.4.3 RQ3: Specification Accuracy and Contract Coverage. To answer RQ3, we introduce $\text{ContractCoverage}@k$. While measuring absolute semantic completeness across all execution paths is practically unfeasible, Contract Coverage serves as a pragmatic, defect-driven proxy evaluating whether the generated contract captures the pre- or post-condition violated by a historical bug. This removes test generation capability as a confounding variable when assessing oracle quality. We note that this metric carries two specific limitations: it requires a ground-truth fix and reproducible bug, and it assesses contract boundaries only relevant to the target defect rather than the entire component’s behavioral spectrum.

An LLM-as-a-judge evaluates the specifications. Similar to the procedure in §4.4.2, the judge is invoked 5 times per evaluation, and the final decision regarding Contract Coverage is determined via majority vote. To ensure a comprehensive, high-fidelity evaluation of contract alignment, the judge is provided with: (1) the bug description, (2) the developer’s commit message (CL description) capturing the human developer’s intent, (3) the unified, ground-truth human fix diff of the source files, and (4) the generated specification. The judge performs the following tasks:

Contract Coverage Assessment: Determining if the generated specification explicitly documents the behavioral constraint that was violated by the historical bug and addressed by the fix.

Taxonomy Classification: Categorizing the nature of the documented contract (e.g., distinguishing between low-level safety specifications like memory allocation limits, and high-level behavioral specifications like protocol state compliance).

Failure Diagnosis: For bugs that were *not* detected, classifying the root cause either as a “Specification Generation Failure”, where the testing oracle was missing the contract entirely, or a “Test Generation Failure”, where the contract was identified but a compiling, fault-revealing test was not generated.

5 Evaluation Results

In this section, we present results from our empirical study.

5.1 RQ1: Quantitative Effectiveness

We evaluate whether grounding an agent in behavioral specifications improves its capability to detect real-world bugs and achieve higher structural coverage. We analyze these aspects using two primary dimensions: fault detection capability ($\text{detect}@k$) and structural coverage (line and branch coverage) across multiple independent runs ($k \in [1, 5]$).

5.1.1 Test Suite Pass Rate ($\text{pass}@k$). We first measure $\text{pass}@k$ as a sanity check, which measures the rate at which the agent authors syntactically valid and contextually correct test suites that compile and pass entirely on the fixed version of the source code. This metric acts as a necessary baseline to ensure both agents are capable of producing valid code before evaluating their bug detection capabilities. Table 1 reports the pass rates for both baseline and spec-driven agents as k increases from 1 to 5.

As shown in Table 1, both configurations can generate working test suites. At $k = 1$, both agents achieve a pass rate of over 94%, which rapidly scales to 98.9% at $k = 5$. The negligible difference between the approaches (-0.2% at $k = 1$ and 0.0% at $k = 5$) confirms both agents possess an equally robust baseline for code synthesis

Table 1: Test suite $\text{pass}@k$. Both agents achieve similarly high rates, validating that spec-driven reasoning does not degrade code generation performance.

k	Baseline Agent	Spec-Driven Agent	Difference
1	94.4%	94.2%	-0.2%
2	98.0%	97.8%	-0.2%
3	98.4%	98.3%	-0.1%
4	98.7%	98.7%	0.0%
5	98.9%	98.9%	0.0%

and as a result improvements observed in branch coverage and bug detection are driven by the quality of the semantic contract guidance rather than code generation proficiency.

5.1.2 Fault Detection Rate ($\text{detect}@k$). The $\text{detect}@k$ metric estimates the probability of successfully detecting a historical bug within k independent test generation runs. A bug is considered detected if the generated test suite compiles and passes on the fixed version of the code but triggers test execution failures on the buggy version. Table 2 summarizes the bug detection rates for both agents as k increases from 1 to 5, along with their bootstrapped 95% confidence intervals (CI) and McNemar’s test p -values.

As shown in Table 2, the Spec-Driven Agent consistently outperforms the Baseline Agent at every value of k . At $k = 1$, the Spec-Driven Agent achieves a 41.1% detection rate, representing a 4.2% absolute improvement over the baseline (36.9%). Notably, the performance gap widens as the number of runs increases. At $k = 5$, the Spec-Driven Agent reaches a detection rate of 63.2%, yielding a 9.8% absolute improvement over the baseline agent (53.4%). This widening gap suggests that the spec-driven framework leverages additional execution budgets to guide the agent towards exploring diverse path spaces and uncovering subtle bugs.

Progress on More Challenging Bugs. It is also worth noting the relatively wide 95% confidence intervals observed for both approaches in Table 2. This broad spread is primarily attributable to the high variance in bug detection difficulty within our dataset. Rather than a uniform probability of detection across all faults, the per-bug detection rates exhibit a heavily bi-modal distribution: individual historical bugs tend to be either consistently caught across almost all independent runs (low difficulty) or almost never caught in any run (high difficulty). Specifically, in Baseline Agent, 65.6% of the bugs (59 out of 90) fall into these two extreme categories of 0% or 100% detection rates. The Spec-Driven Agent reduces this bi-modality slightly to 57.8% (52 out of 90) by shifting several previously undetected (0%) bugs into the intermediate region (20% to 80% detection rates) where success is conditional. This shift indicates that the Spec-Driven Agent makes progress on more challenging bugs, converting them from completely undetected (0% detection rate) to non-deterministically detected (20% to 80% detection rate). This intrinsic polarization in problem complexity naturally inflates the variance of the overall mean estimation across the sample, resulting in the wider confidence bounds.

Uniquely Detected Bugs. To understand the uniqueness of the bugs detected by each agent, we analyzed the overlap of successful

Table 2: Fault detection rate (detect@ k) with 95% confidence intervals and McNemar’s test p -values.

k	Baseline Agent	Spec-Driven Agent	Difference	McNemar’s p -value
1	36.9% [28.4%, 45.3%]	41.1% [32.9%, 49.3%]	+4.2%	$p=0.3075$
2	45.3% [36.2%, 54.6%]	51.3% [42.2%, 60.3%]	+6.0%	$p=0.4545$
3	49.4% [39.6%, 59.2%]	56.6% [47.0%, 65.9%]	+7.2%	$p=0.0574$ (Marginal)
4	51.8% [41.8%, 61.8%]	60.5% [50.7%, 70.0%]	+8.7%	$p=0.0352$ (Significant)
5	53.4% [43.3%, 63.3%]	63.2% [53.3%, 73.3%]	+9.8%	$p=0.0352$ (Significant)

detections at $k = 5$. Both agents successfully detected a shared subset of 45 bugs. The Spec-Driven Agent uniquely detected 12 bugs that the Baseline Agent missed, while the Baseline Agent uniquely detected 3 bugs that the Spec-Driven Agent missed.

Statistical Significance. We applied McNemar’s test to evaluate the statistical significance of the difference in bug detection rates at each k , as shown in Table 2. At lower execution budgets ($k = 1$ and $k = 2$), the differences are not statistically significant ($p > 0.05$). However, as the execution budget increases, the differences become statistically meaningful. At $k = 3$, we observe marginal significance ($p = 0.0574$). At higher execution budgets of $k = 4$ and $k = 5$, the Spec-Driven Agent’s improvement over the Baseline Agent becomes statistically significant, yielding a p -value of 0.0352 ($p < 0.05$) in both cases. This demonstrates that grounding the agent’s reasoning in explicit behavioral contracts yields superior (statistically significant at higher execution budgets) fault-revealing capability compared to direct prompt-to-code synthesis.

5.1.3 Structural Coverage. We evaluate structural coverage using the average line and branch coverage of the target source file achieved by the generated test suites on the fixed code version. For each target source file, the coverage percentage per run is defined as the number of executed lines (or branches) over the number of instrumented (executable) lines (or branches) in that file. For each bug, we compute the mean of this coverage across all $k = 5$ runs. Table 3 presents the overall mean coverage results across all bugs, along with their 95% confidence intervals (estimated via bug-level bootstrapping) and Wilcoxon signed-rank test.

The Spec-Driven Agent achieves slightly lower average line coverage (74.4% vs 74.8%, -0.4%) but notably higher average branch coverage (48.9% vs 46.4%, +2.5% improvement). To assess statistical significance, we performed a Wilcoxon signed-rank test on the paired coverage distributions, as shown in Table 3, and find that the branch coverage difference is highly statistically significant (p -value= 0.0034), while the difference in line coverage is not statistically significant (p -value= 0.3659).

This indicates that the spec-driven framework systematically drives the agent to cover more complex control flow structures (branches), which are crucial for robust validation, rather than simply inflating line coverage by executing straight-line paths.

5.2 RQ2: Qualitative Test Rigor

We compare the 83 successfully generated tests from the Spec-Driven Agent against both the test suites generated by the Baseline Agent and the ground truth, developer-written test suites using an

LLM-as-a-Judge. The judge demonstrated high internal reliability, achieving high self-agreement ($\geq 90\%$) and an extremely low tie rate ($\leq 5.6\%$) across 5 independent evaluation runs. Figures 2 and 3 visualize the qualitative evaluation distribution.

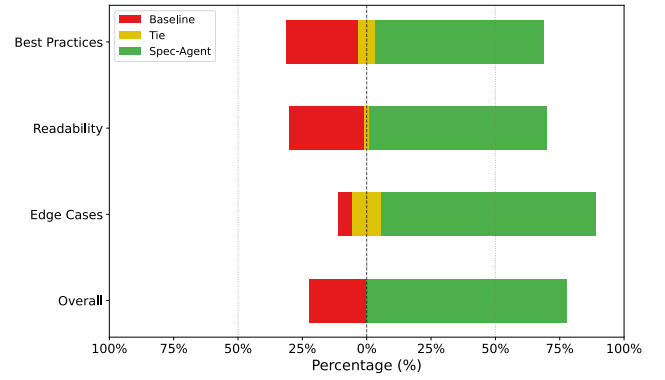


Figure 2: Test suites generated by Baseline Agent vs. Spec-Driven Agent. The Spec-Driven Agent produces tests that are consistently judged as higher quality.

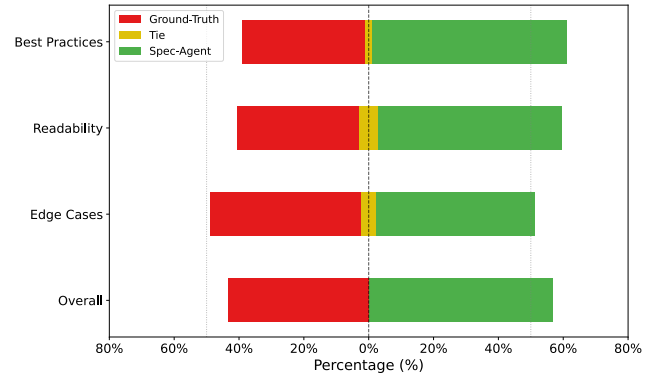


Figure 3: Developer-written test suites vs. test suites generated by Spec-Driven Agent. Spec-Driven Agent generates test suites at a quality-level competitive with the original human-written ones, judged via LLM-as-a-Judge.

5.2.1 Qualitative Evaluation Results. We first analyze test suites from the Spec-Driven Agent against those from the Baseline Agent, then against the original test suites written by human developers.

Table 3: Average line and branch coverage with 95% confidence intervals.

Metric	Baseline Agent	Spec-Driven Agent	Difference	Wilcoxon p-value
Line Coverage	74.8% [69.6%, 79.8%]	74.4% [68.6%, 79.9%]	-0.4%	p=0.3659 (N.S.)
Branch Coverage	46.4% [39.6%, 53.3%]	48.9% [41.4%, 56.4%]	+2.5%	p=0.0034 (Significant)

Spec-Driven Agent vs. Baseline Agent. As shown in Figure 2, test suites from Spec-Driven Agent overwhelmingly outperformed the Baseline Agent across all dimensions on 77.8% of the cases. The judge consistently preferred the test suites from Spec-Driven Agent for granular behavioral isolation, idiomatic framework utilization (e.g., avoiding reflection and over-mocking), and contract-aligned naming conventions (e.g., `Method_State_Outcome`). Notably, test suites from Spec-Driven Agent excelled in edge case coverage (83.3%), systematically validating boundary limits, missing optionality, and error paths that the baseline completely missed. The higher quality of test suites from Spec-Driven Agent correlated empirically with its 9.8 percentage point advantage in bug detection (§5.1.2).

Spec-Driven Agent vs. Developer-Written Test Suites. Compared to test suites from human developers governed by strict style guides and code review, Spec-Driven Agent generated test suites on par or marginally better, being overall superior in 56.7% of cases, as shown in Figure 3.

- **Agent Strengths:** The Spec-Driven Agent generated narrower, high-precision assertions and minimized boilerplate through localized factory helpers. Grounded by contracts, it systematically verified negative error paths dictated by the contract. It also favored robust concurrency constructs (e.g., `absl::Notification`) over the non-deterministic waits occasionally found in human tests.
- **Agent Weaknesses:** Human developers excelled at designing exhaustive validation for complex algorithmic scenarios (e.g., intricate state transitions and timezones) that the agent addressed superficially. Furthermore, humans strictly adhered to public API encapsulation, whereas the Spec-Driven Agent occasionally resorted to reflection-based hacks (e.g., `invokePrivateMethod`) to bypass encapsulation.

In summary, across all evaluated qualitative dimensions, the Spec-Driven Agent consistently outperforms the direct generation baseline, while achieving parity with—and occasionally marginally exceeding—the strict rigor of developer-written tests.

5.3 RQ3: Specification Accuracy

We evaluate how accurately the agent-generated semi-formal specifications establish valid testing oracles that capture the violated behavioral contracts. We run the analysis at the attempt level across all $5 \times 90 = 450$ runs.

5.3.1 Contract Coverage and Bug Detection. Our analysis of the generated specifications reveals that at a single attempt ($k = 1$), the Spec-Driven Agent achieves a `ContractCoverage@1` of 61.1% [53.1%, 70.7%], successfully documenting the exact behavioral contract violated in the majority of cases. This coverage rate scales robustly

with multiple attempts, rising to 69.7% [61.3%, 78.6%] at $k = 2$, and peaking at a `ContractCoverage@5` of 78.9% [70.0%, 86.7%].

To evaluate the utility of specifications as testing oracles, we constructed a 2×2 contingency table consisting of binary outcomes of contract coverage and bug detection across all attempts. Because both metrics are binary variables, we evaluate association using the Phi coefficient and assess significance via Fisher’s exact test (Fisher’s exact test $p = 3.62 \times 10^{-14}$, Phi coefficient $\phi = 0.35$), and we find a statistically significant moderate correlation:

- **With Contract Coverage:** When the generated specification successfully documented the violated contract, the agent generated a bug-detecting test suite in 54.9% of cases (151 out of 275 runs).
- **Without Contract Coverage:** When the specification missed the contract, the detection rate dropped to 19.4% (34 out of 175 runs).

This finding provides strong empirical support for our core hypothesis: *Grounding the agent’s reasoning in explicit behavioral contracts significantly improves automated test generation.* Specifically, when a contract is established in the intermediate specification artifact, the synthesis phase effectively translates it into a fault-revealing test (54.9%). Conversely, without a contract established, the agent is forced to generate tests without semantic direction, resulting in a lower baseline success rate of 19.4%.

The remaining 34 runs that detected the bug despite lacking contract coverage represent cases where generic structural coverage goals or simple crash assertions (such as checking for unhandled exceptions) happened to trigger the bug by chance, rather than through targeted oracle-driven validation.

5.3.2 Specification Generation Failure. We consider the 141 runs as that failed to detect the bug and where the generated specification did not cover the buggy behavior as *spec generation failures*. We categorized these failures via an LLM judge into five patterns:

- **Omission of Methods (35):** Completely failing to identify and document a specific method, helper, or RPC handler.
- **Missing Error Handling (32):** Documenting only the happy path, missing exceptions, fallbacks, and edge cases.
- **Omitted Data Transformations (27):** Missing dynamic field mappings, string manipulations, math calculations.
- **Abstraction of Constants (25):** Acknowledging constants or formats but abstracting exact values or key mappings.
- **Missing Execution Constraints (22):** Missing precise execution ordering, negative constraints, boundary conditions.

5.3.3 Test Generation Failure. We consider the 110 runs where the contract is deemed covered but the bug is not detected as *test generation failures*. They are semantic failures where the test suite passed the fixed code but failed to catch the bug, and do not include

agent framework or compilation failures (14 runs). We categorize failures in these runs into five patterns:

- **Inadequate Input Data (37):** Providing overly simplistic, trivial, or happy-path inputs without necessary edge-case triggers (e.g., null values, specific timezones).
- **Omitted Scenarios (36):** Ignoring explicit specification statements and failing to generate tests for specific branches or methods.
- **Weak or Missing Assertions (20):** Executing the buggy path but failing to strictly verify state mutations or using overly broad assertions.
- **Flawed Test Logic (12):** Masking bugs with internal test flaws, such as race conditions, over-mocking, or missing trigger actions.
- **Invalid Test Code (5):** Generating syntactically incorrect code or targeting non-existent signatures, resulting in compilation failures.

5.4 RQ4: Cost-Efficiency

While grounding agents in explicit behavioral contracts yields substantial improvements in both structural coverage (§5.1) and qualitative test rigor (§5.2), these benefits must be weighed against the incurred computational cost. To evaluate this trade-off, we perform a detailed analysis on the token consumption.

Table 4 presents the aggregate token consumption for both baseline and spec-driven agent configurations across all 5 runs of the evaluated dataset, split by input (prompt) and output (generation) tokens, along with the resulting bug-detection yield.

Table 4: Aggregate cost-efficiency metrics.

Metric	Baseline	Spec-Driven	Overhead / Ratio
Total Tokens Consumed	243.9M	336.7M	1.38× (+38.0%)
Input Tokens	224.4M	305.7M	1.36× (+36.2%)
Output Tokens	19.5M	31.0M	1.59× (+59.1%)
Unique Bugs Detected ($k=5$)	48	57	+9 (+18.8%)
Tokens per Unique Bug Detected	5.1M	5.9M	1.16× (+16.2%)

As shown in Table 4, the Spec-Driven Agent consumed a total of 336.6M tokens, representing a 38.0% overhead over the Baseline Agent’s 243.9M tokens. This overhead is a direct consequence of the multi-phase spec-driven test generation framework: the test generation phase (§3.3) carries the overhead of prepending the specification generated from the specification extraction phase (§3.1) to the agent’s context window.

Interestingly, while input tokens increased by 36.2% (reflecting the larger prompt contexts containing specifications), output tokens exhibited a higher relative increase of 59.1% (30.9M vs 19.4M). This increase is driven by the Spec-Driven Agent (1) must output the comprehensive behavioral specification in markdown, and (2) synthesize more robust, thorough, and consequently larger test suites containing extensive edge-case coverage while being guided by the specification.

6 Threats to Validity

Construct Validity. Our evaluation relies on an LLM-as-a-Judge (Gemini 3.1 Pro) for assessing qualitative test rigor (§5.2) and for

computing contract coverage (§5.3). A known threat is that LLMs can exhibit classification noise, verbosity bias, and self-preference bias. We mitigate this by (1) deliberately using a larger, more capable Gemini 3.1 Pro model to evaluate the outputs of a smaller Gemini 3.0 Flash model which helps reduce self-preference bias, and (2) executing the LLM judge 5 independent times for every evaluation pair and computing the final decision via a strict majority vote. This ensemble approach smooths classification noise and ensures consistent self-agreement across the independent runs.

Internal Validity. A pervasive threat when evaluating frontier AI models on historical software engineering datasets is data contamination: the risk that the model encountered the exact evaluation bugs during its pre-training phase, leading to memorization rather than true reasoning. However, in our evaluation, both the baseline and the Spec-Driven Agent utilize the exact same underlying LLM. Therefore, any observed performance improvements are strictly due to the enhanced reasoning capabilities unlocked by the specification rather than data contamination.

Since our evaluation relies on a single set of LLMs (Gemini), the absolute performance metrics are inherently tied to this version’s underlying code-reasoning capabilities, so using different or future LLMs may yield different absolute bug detection and compilation rates. We use the same model set and agentic harness for both evaluated agents to mitigate this threat.

Evaluating bug detection on historical bugs is an established protocol for regression testing. However, we acknowledge that developers want to find new bugs in practical production environments. We focus on understanding the feasibility of semi-formal specification framework in helping the agent to recover bug-catching contracts, by evaluating on historical bugs. Given its effectiveness illustrated in this paper, future work can explore using the framework to aid agentic discovery of new bugs.

External Validity. While our dataset is multi-lingual (C++, Java, Python, Go) and covers diverse domains, all historical bugs were sourced from Google’s monorepo, which represents a single large-scale technology organization with specific engineering standards and style guides. Consequently, our metrics must be interpreted in the context of this specific environment. While the exact detection and coverage rates may vary across different corporate or open-source codebases, the underlying methodology of using semi-formal contracts as a cognitive scaffold for agentic exploration is fundamentally language- and platform-agnostic. Additionally, our evaluation scope is currently limited to bugs whose fixes modify a single source file to establish cleaner evaluation boundaries and straightforward comparison.

7 Related Work

We discuss related work in program verification broadly, LLM-based program reasoning, and test generation specifically.

Hoare logic [21] introduced a compositional approach to formally reasoning about program behavior, based on pre- and post-conditions, setting the foundations for modern-day program verification. The seminal Eiffel programming language [26] built on these foundations, introducing design-by-contract, which allows developers to clearly document each component’s requirements

and responsibilities by co-locating their logical assertions with their implementation and automatically checking these at runtime. Advances in automated reasoning tools and modern verification-aware programming languages, like Dafny [25] and F^* [37], allow developers to statically verify that their implementations satisfy their logical properties. Similarly, Lean [14] and Rocq [8] can be used to write rigorous, machine-checkable proofs, allowing computer scientists and mathematicians to prove properties about their programs. Recently, Lean in particular has seen substantial uptake in the AI for math and code research community, with agentic approaches tackling challenges such as autoformalization [45].

Similar to this line of work, we use pre/post conditions to specify key program properties. However, our focus is on using these properties to generate robust test suites. We employ an agentic approach to generating the specifications and the test suite, and do not expect users to manually write additional specification properties, nor do we aim to prove full functional correctness (or other important properties, like safety) of the underlying program.

More generally, LLMs have proven to be capable of reasoning about varying types of code properties [42]. For example, CRUXEval [19] introduced a benchmark to evaluate LLM’s reasoning about execution (both forward and backwards). Similarly, RubberDuckBench [29] constructed more complex code reasoning questions. Given the effectiveness of compositional reasoning in software, research has focused on invariant mining and using LLMs to generate preconditions [24], post-conditions [15], and invariants. Unlike traditional dynamic invariant inference (e.g., Daikon) which captures actual execution traces (including bugs) in rigid mathematical formulas, LLMs leverage contextual signals (docstrings, naming) to capture developer intent in highly readable, semi-formal natural language contracts that generalize across multi-language codebases without runtime setup. For example, HoarePrompt [9] showed LLMs can produce natural language counterparts to these logical assertions and use them to identify faulty programs that fail to satisfy their original intent. Ugare and Chandra [39] show that LLM-based agents can build up semi-formal arguments for program equivalence, fault localization, and code question answering. SpecBench [20] recently introduced a new benchmark focused on agentic reasoning for program specifications.

In contrast to this work, we are not focused on general program reasoning, such as reasoning about intermediate states or answering code property questions. Our agentic workflow is focused on test generation, and we use specifications as a way to produce more robust tests. While our agent may choose to perform semi-formal reasoning while generating the specifications or the tests, it is not constrained to this particular approach. Finally, we evaluate our approach in the context of a large industrial codebase.

Another line of related work is test generation. Non-learning-based techniques have focused on generating tests that maximize coverage [16, 32], while learning-based techniques further consider code naturalness of generated tests [31, 34, 38, 40]. Recent LLM-based techniques have explored feedback-driven test generation [4, 12, 30], change-description guided behavior test generation [33], bug reproduction test generation [1, 2, 11], and test suite generation [22]. In contrast to the prior work, we explicitly break down test generation into a specification generation phase and a test generation phase, with the former grounding the latter. Moreover, we

focus on evaluating test generation for industrial code, rather than in the context of program repair or new code changes.

Concurrently with our work, [35] introduced DeepTest, which generates tests based on a specification extracted by an agent. While DeepTest builds out its specification by extracting call graphs and analyzing expected relationships (incorporating additional knowledge such as comments and change descriptions), our approach strictly structures the specification as a semi-formal, natural language artifact (`.spec.md`) centered on explicit pre- and post-conditions. Our work is complementary to DeepTest, as we evaluate the performance of this general pattern of grounding test generation on mined specifications in industrial code.

General-purpose cognitive scaffolding, such as Chain-of-Thought (CoT) reasoning, is already a default behavior in LLMs and is natively utilized by our Baseline Agent. While our work focuses on evaluating semi-formal contracts as a specialized alternative, future work will explore a direct empirical comparison with other structured cognitive scaffolding techniques, such as explicit high-level test planning, pseudocode drafting, or execution-trace modeling. Future work can also complement contract coverage with precision-oriented metrics to identify over-specifications or hallucinations, and compare semi-formal contracts against traditional static analysis and dynamic invariant miners.

8 Conclusions

In this paper, we proposed Spec-Driven Test Generation, a novel agentic framework that bridges Design by Contract principles with automated software validation. By instructing agents to explicitly reason about and document a component’s behavioral boundaries in a semi-formal specification before generating test code, our approach provides a critical cognitive scaffold that elevates the quality and fault-revealing capability of the generated test suites, while guiding a systematic state-space exploration.

Our empirical evaluation on a dataset of 90 real-world, historical bugs from Google’s production systems demonstrates that grounding agents in these self-generated specifications yields statistically significant improvements. At $k = 5$ runs, the Spec-Driven Agent achieved a 63.2% bug detection rate (+9.8% absolute improvement over the baseline) and a significant increase in branch coverage. Furthermore, qualitative evaluations by an LLM-as-a-Judge showed that spec-driven test suites are overall superior to baseline-generated suites in 77.8% of cases, and even outperform or match expert developer-written tests in 56.7% of cases. We also introduced the `ContractCoverage@k` metric to evaluate the semantic completeness of the generated specifications. We observed a significant correlation between achieving this coverage and downstream bug detection success, highlighting the utility of the specification as a guiding testing oracle.

References

- [1] Toufique Ahmed et al. 2025. Heterogeneous Prompting and Execution Feedback for SWE Issue Test Generation and Selection. *arXiv preprint arXiv:2508.06365* (2025).
- [2] Toufique Ahmed et al. 2025. Otter: Generating tests from issues to validate SWE patches. *arXiv preprint arXiv:2502.05368* (2025).
- [3] M. M. Almasi, H. Hemmati, G. Fraser, A. Arcuri, and J. Benefelds. 2017. An Industrial Evaluation of Unit Test Generation: Finding Real Faults in a Financial Application. In *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*. 263–272.

- doi:10.1109/icse-seip.2017.27
- [4] J. Altmayer Pizzorno and E. D. Berger. 2025. Coverup: Effective high coverage test generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2897–2919. doi:10.1145/3729398
 - [5] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. 2014. The oracle problem in software testing: A survey. *IEEE transactions on software engineering* 41, 5 (2014), 507–525. doi:10.1109/TSE.2014.2372785
 - [6] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2016. The Satisfiability Modulo Theories Library (SMT-LIB). www.SMT-LIB.org.
 - [7] Kent Beck. 2003. *Test-driven development: by example*. Addison-Wesley Professional.
 - [8] Y. Bertot and P. Castéran. 2004. *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*. Springer Science & Business Media. doi:10.1007/978-3-662-07964-5
 - [9] Dimitrios Stamatiou Bouras et al. 2025. HoarePrompt: Structural Reasoning About Program Correctness in Natural Language. *arXiv preprint arXiv:2503.19599* (2025).
 - [10] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. O. D. O. Pinto, J. Kaplan, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
 - [11] Runxiang Cheng et al. 2026. Dynamic Cogeneration of Bug Reproduction Test in Agentic Program Repair. *arXiv preprint arXiv:2601.19066* (2026). doi:10.1145/3803437.3805237
 - [12] A. M. Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais. 2024. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology* 171 (2024), 107468. doi:10.1016/j.infsof.2024.107468
 - [13] Leonardo De Moura, Soonho Kong, Jeremy Avigad, Floris Van Doorn, and Jakob von Raumer. 2015. The Lean theorem prover (system description). In *International Conference on Automated Deduction*. Springer, 378–388. doi:10.1007/978-3-319-21401-6_26
 - [14] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer. 2015. The Lean Theorem Prover (System Description). In *Automated Deduction – CADE-25*. Springer, 378–388. doi:10.1007/978-3-319-21401-6_26
 - [15] Madeline Endres et al. 2024. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1889–1912. doi:10.1145/3660791
 - [16] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. doi:10.1145/2025113.2025179
 - [17] Gemini Team, Google. 2025. Gemini 3 Flash Model Card. <https://deepmind.google/models/model-cards/gemini-3-flash/>. Accessed: June 2026.
 - [18] Gemini Team, Google. 2026. Gemini 3.1 Pro Model Card. <https://deepmind.google/models/model-cards/gemini-3-1-pro/>. Accessed: June 2026.
 - [19] Alex Gu et al. 2024. Cruxeval: A benchmark for code reasoning, understanding and execution. *arXiv preprint arXiv:2401.03065* (2024).
 - [20] Grant Hamblin et al. 2026. SpecBench: Evaluating Specification-Level Reasoning for Software Engineering LLM Agents. *arXiv preprint arXiv:2605.30314* (2026).
 - [21] Charles Antony Richard Hoare. 1969. An axiomatic basis for computer programming. *Commun. ACM* 12, 10 (1969), 576–580. doi:10.1145/363235.363259
 - [22] Kush Jain and Claire Le Goues. 2025. Testforge: Feedback-driven, agentic test suite generation. *arXiv preprint arXiv:2503.14713* (2025).
 - [23] R. Just, D. Jalali, and M. D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. doi:10.1145/2610384.2628055
 - [24] Daragh King, Vasileios Koutavas, and Laura Kovács. 2025. LLM-Based Generation of Weakest Preconditions and Precise Array Invariants. In *2025 IEEE/ACM 13th International Conference on Formal Methods in Software Engineering (FormalISE)*. IEEE. doi:10.1109/formalise66629.2025.00016
 - [25] K. R. M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16)*. Springer, 348–370. doi:10.1007/978-3-642-17511-4_20
 - [26] Bertrand Meyer. 1988. Eiffel: A language and environment for software engineering. *Journal of Systems and Software* 8, 3 (1988), 199–246. doi:10.1016/0164-1212(88)90022-2
 - [27] B. Meyer. 1992. Applying 'design by contract'. *Computer* 25, 10 (1992), 40–51. doi:10.1109/2.161279
 - [28] B. Meyer. 2018. Seven Principles of Software Testing. *Software Technology: 10 Years of Innovation in IEEE Computer* (2018), 202–207. doi:10.1109/mc.2008.306
 - [29] Ferida Mohammad et al. 2026. RubberDuckBench: A Benchmark for AI Coding Assistants. *arXiv preprint arXiv:2601.16456* (2026).
 - [30] N. Mündler, M. N. Müller, J. He, and M. Vechev. 2024. Swt-bench: Testing and validating real-world bug-fixes with code agents. In *Advances in Neural Information Processing Systems*, Vol. 37. 81857–81887.
 - [31] P. Nie, R. Banerjee, J. J. Li, R. J. Mooney, and M. Gligoric. 2023. Learning deep semantics for test completion. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2111–2123. doi:10.1109/icse48619.2023.00178
 - [32] Rahul Pandita et al. 2010. Guided test generation for coverage criteria. In *2010 IEEE international conference on software maintenance*. IEEE. doi:10.1109/icsm.2010.5609565
 - [33] Michael Pradel. 2025. Testora: Using Natural Language Intent to Detect Behavioral Regressions. *arXiv preprint arXiv:2503.18597* (2025).
 - [34] N. Rao, K. Jain, U. Alon, C. Le Goues, and V. J. Hellendoorn. 2023. CAT-LM training language models on aligned code and tests. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 409–420. doi:10.1109/ase56229.2023.00193
 - [35] RISE MSR. 2026. DeepTest Blog Post. [urlhttps://risemr.github.io/blog/2026-05-04-deeptest/](https://risemr.github.io/blog/2026-05-04-deeptest/).
 - [36] S. Shamshiri, R. Just, J. M. Rojas, G. Fraser, P. McMinn, and A. Arcuri. 2015. Do Automatically Generated Unit Tests Find Real Faults? An Empirical Study of Effectiveness and Challenges (T). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 201–211. doi:10.1109/ase.2015.86
 - [37] Nikhil Swamy et al. 2016. Dependent types and multi-monadic effects in F. In *Proceedings of the 43rd annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. doi:10.1145/2837614.2837655
 - [38] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020).
 - [39] Shubham Ugare and Satish Chandra. 2026. Agentic code reasoning. *arXiv preprint arXiv:2603.01896* (2026).
 - [40] C. Watson, M. Tufano, K. Moran, G. Bavota, and D. Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 1398–1409. doi:10.1145/3377811.3380429
 - [41] Titus Winters, Tom Manshreck, and Hyrum Wright. 2020. *Software engineering at google: Lessons learned from programming over time*. "O'Reilly Media, Inc.". <https://abseil.io/resources/swe-book>
 - [42] Dayu Yang et al. 2025. Code to think, think to code: A survey on code-enhanced reasoning and reasoning-driven code intelligence in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.
 - [43] John Yang et al. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. 50528–50652.
 - [44] X. Yin, C. Ni, X. Xu, and X. Yang. 2024. What You See Is What You Get: Attention-based Self-guided Automatic Unit Test Generation. *arXiv preprint arXiv:2412.00828* (2024). doi:10.1109/icse55347.2025.00105
 - [45] Lan Zhang, Marco Valentino, and André Freitas. 2025. Autoformalization in the wild: Assessing LLMs on real-world mathematical definitions. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Received 2026-06-29; accepted 2026-07-16